

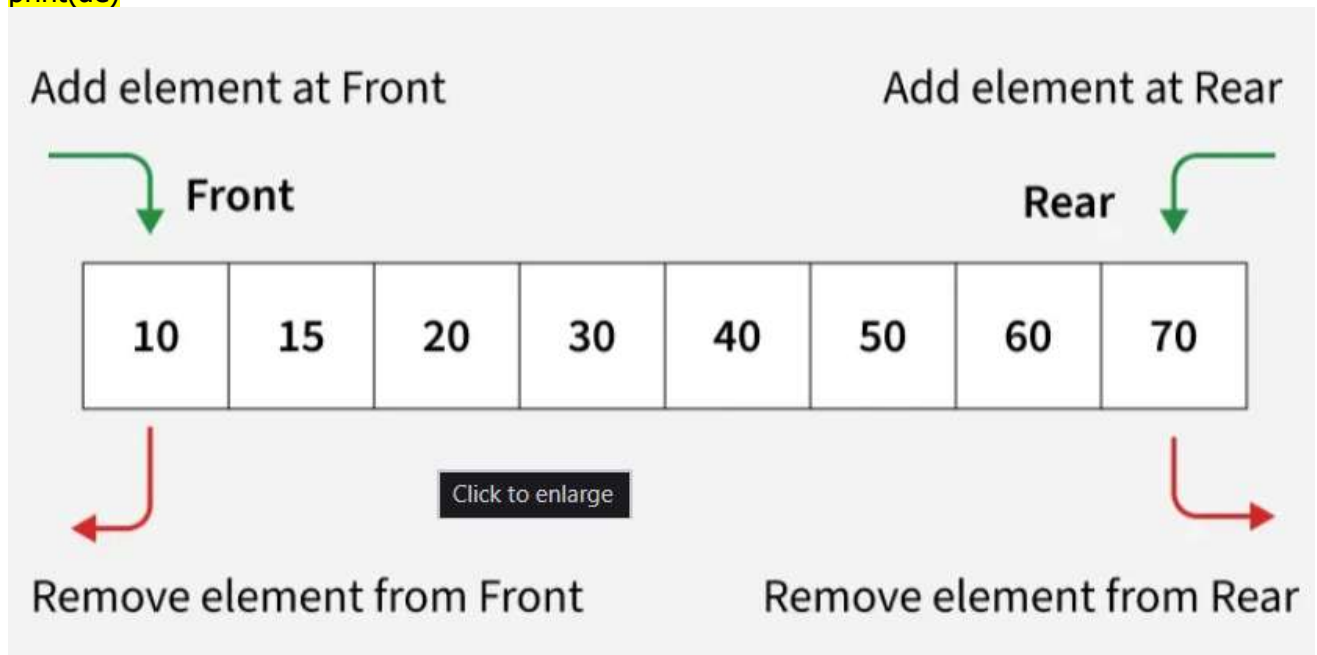
Deque

Tuesday, May 5, 2026 11:16 AM

It is a special type of data structure that allows to add and remove elements from both ends efficiently.

This makes it useful in applications like task scheduling, sliding window problems and real-time data processing.

```
from collections import deque
de = deque(['name','age','DOB'])
print(de)
```



- It supports $O(1)$ time for adding/removing elements from both ends
- It is more efficient than lists for front-end operations
- It can function as both FIFO and LIFO.
- It offers `appendleft()`, `popleft()` and `rotate()`.

There are two types of restricted deques:

- Input restricted Deque: input is limited at one end while deletion is permitted at both ends.
- Output restricted Deque: output is limited at one end and insertion is permitted at both ends.

Methods:

- Append: adds `x` to the right end of the deque
- Appendleft: adds `x` to the left end
- Extend(iterable): adds all elements from the iterable to the right end
- Extendleft(iterable)
- Remove: removes the first occurrence of the specified value from the deque.
- Pop(): removes an element from the right end
- Popleft()
- Clear: removes all elements

```
from collections import deque
dq = deque([10, 20, 30])
```

```
# Add elements to the right
dq.append(40)
```

```
# Add elements to the left
dq.appendleft(5)
```

```
# extend(iterable)
```

```

dq.extend([50, 60, 70])
print("After extend([50, 60, 70]):", dq)

```

```

# extendleft(iterable)
dq.extendleft([0, 5])
print("After extendleft([0, 5]):", dq)

```

```

# remove method
dq.remove(20)
print("After remove(20):", dq)

```

```

# Remove elements from the right
dq.pop()

```

```

# Remove elements from the left
dq.popleft()
print("After pop and popleft:", dq)

```

```

# clear() - Removes all elements from the deque
dq.clear() # deque: []
print("After clear():", dq)

```

- Count(value): counts the number of occurrences of a specific element in the deque
- Rotate(n): rotates the deque by n steps. Positive n rotates to the right and negative n rotates to the left.
- Reverse: reverses the order of the elements in the deque.

from collections import deque

```

# Create a deque
dq = deque([10, 20, 30, 40, 50, 20, 30, 20])

```

```

# 1. Counting occurrences of a value
print(dq.count(20)) # Occurrences of 20
print(dq.count(30)) # Occurrences of 30

```

```

# 2. Rotating the deque
dq.rotate(2) # Rotate the deque 2 steps to the right
print(dq)

```

```

dq.rotate(-3) # Rotate the deque 3 steps to the left
print(dq)

```

```

# 3. Reversing the deque
dq.reverse() # Reverse the deque
print(dq)

```

append(x)	Adds x to the right end of the deque.	O(1)
appendleft(x)	Adds x to the left end of the deque.	O(1)
pop()	Removes and returns an element from the right end of the deque.	O(1)
popleft()	Removes and returns an element from the left end of the deque.	O(1)
extend(iterable)	Adds all elements from iterable to the right end of the deque.	O(k)

append(x)	Adds x to the right end of the deque.	O(1)
appendleft(x)	Adds x to the left end of the deque.	O(1)
pop()	Removes and returns an element from the right end of the deque.	O(1)
popleft()	Removes and returns an element from the left end of the deque.	O(1)
extend(iterable)	Adds all elements from iterable to the right end of the deque.	O(k)
extendleft(iterable)	Adds all elements from iterable to the left end of the deque (reversed order).	O(k)
remove(value)	Removes the first occurrence of value from the deque. Raises ValueError if not found.	O(n)
rotate(n)	Rotates the deque n steps to the right. If n is negative, rotates to the left.	O(k)
clear()	Removes all elements from the deque.	O(n)
count(value)	Counts the number of occurrences of value in the deque.	O(n)
index(value)	Returns the index of the first occurrence of value in the deque. Raises ValueError if not found.	O(n)
reverse()	Reverses the elements of the deque in place.	O(n)